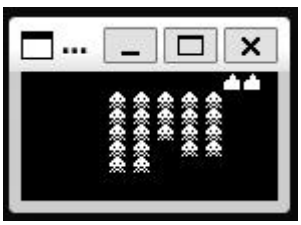
오픈소스 기반 반도체 칩 종합 설계

권효재

이번에는 교수님께서 강의에서 알려주신 방법대로 파일을 업데이트하고 FPGA와 pico를 다시 설정한 뒤 시뮬레이션부터 프로토타이핑까지 진행했다. 지난주에는 시뮬레이션과 FPGA 합성까지 성공했지만, oled에는 테두리와 점 몇개만 출력되고 게임이 제대로 나타나지 않았다.

- Q14 파일 갱신과 시뮬레이션

먼저 기존 Q14 파일에 문제가 있는지 확인하기 위해 교수님의 깃허브 저장소를 새로 clone했다. 기존 프로젝트를 덮어쓰지 않기 위해 별도의 디렉터리에 저장한 뒤 diff 명령으로 파일들을 비교해보니 simulation의 sc_glcd128x64_TLM.h 파일과 ETRI050/simulation의 테스트벤치 관련 파일 5개가 수정되어 있었다. SDL2 화면 출력에 관련된 부분이 수정되어 있었고, 테스트벤치 관련 파일에서는 교수님이 다른 게임에서 복사해 놓으셨던 코드중에서 일부분이 Q14에 맞게 수정되어 있었다. 수정된 파일들을 다른 디렉터리에서 먼저 실행해 보고, 문제가 없는 것을 확인한 뒤 이전 Q14에 복사했다. simulation 디렉터리에서 다시 빌드하고 실행해보니 컴퓨터 SDL2 창에서 외계인 게임이 정상적으로 출력됐다. 화면 갱신도 전보다 훨씬 부드럽게 진행되었다.



Q.14 시뮬레이션

- FPGA 에뮬레이션과 오류 해결

에뮬레이션에서는 먼저 이전에 빌드했던 파일들이 남아 있어 기존 결과물과 새로 빌드한 파일들이 섞이지 않게 FPGA transactor와 pico 펌웨어의 빌드 결과물, 테스트벤치 실행 파일을 정리했다. RTL은 그대로 유지하고 make build_mi, make build_trans, make build를 순서대로 진행했다. pico 펌웨어는 BOOTSEL모드로 진입해서 업로드했다. BOOTSEL과 RESET 버튼을 이용해 RPI-RP2 이동식 드라이브로 전환하고, 윈도우의 D드라이브를 WSL에 마운트해서 빌드된 uf2 파일을 복사했다. FPGA는 quartus로 합성한 뒤 openFPGALoader 1.1.1 버전으로 업로드했다. make run을 실행했는데 컴퓨터의 SDL2 창이 검은 화면으로 나타나고 oled에도 게임이 출력되지 않았다. 처음에는 SDL2 창을 이동시키니 화면이 계단처럼 보이는 렉이 걸렸고, 종료도 되지 않아 kill -9 명령으로 강제 종료했다. pico 리셋버튼을 누르고 다시 진행하는 과정에서는 WSL에서 /dev/ttyACM0가 사라졌고, usbipd attach를 실행하면 Device in error state 오류가 발생했다. 나중에는 pico가 0000:0002로 표시되면서 실패했다. usb 케이블을 분리했다가 다시 연결하니 정상적으로 인식되었고, usbipd attach로 WSL 연결도 복구할 수 있었다. usb 연결을 복구한 뒤 pico 펌웨어와 FPGA 설정 상태를 다시 확인하고 에뮬레이션을 실행했다. 이번에는 Connection established 메시지가 출력되었

고, 컴퓨터의 SDL2와 실제 oled 양쪽에서 외계인 게임이 정상적으로 나타났다. 다만 에뮬레이션 속도가 너무 느려서 키보드 입력을 확인하기 어려울 정도였다. Frame 값은 계속 증가했지만 화면 갱신은 심각하게 느렸다. 교수님께서 보충강의에서 설명하셨듯이 에뮬레이션에서는 컴퓨터의 systemC 테스트벤치와 FPGA 사이에서 pico를 통해 데이터를 주고받는다. 이 과정에서 클럭과 입출력 신호를 usb로 계속 주고받다 보니 통신에 시간이 많이 걸리는 것이었다. 실제 FPGA를 사용했는데도 시뮬레이션보다 훨씬 느리게 동작하는 걸 확인할 수 있었다.



Q.14 에뮬레이션 실패



에뮬레이션 시작전



Q.14 에뮬레이션

```
Info: (I703) tracing timescale unit set: 100 ps (Espace_invaders_engine_glcd.vcd)
Request emulator connection......
Connection established...

Info: (I703) tracing timescale unit set: 100 ps (sc_space_invaders_engine_glcd_TB.vcd)
Frame[9]
```

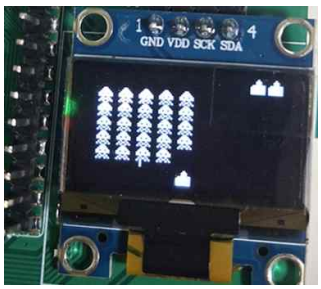
Q.14 에뮬레이션 Frame 증가

- FPGA 프로토타이핑

마지막으로 application 디렉터리에서 프로토타이핑을 진행했다. make build_firmware로 피코 펌웨어를 빌드하고, 전에 진행했던 것처럼 uf2 파일을 직접 업로드했다. 이 과정에서도 /mnt/d 디렉터리가 없어 mount point does not exist 오류가 발생했지만, 디렉터리를 다시 만들고 D드라이브를 마운트해서 해결했다. make build_fpga로 FPGA를 빌드하고, rbf 파일을 생성해 openFPGALoader 1.1.1 버전으로 FPGA에 업로드했다. make run_app을 실행하니 컴퓨터에서는 검은 SDL2 창만 나타나고 키보드 입력 로그만 출력되었다. 처음에는 또 오류가 발생한 줄 알았는데, 실제 oled를 확인해보니 외계인 게임이 정상적으로 실행되고 있었다. 알고 보니 프로토타이핑에서는 컴퓨터가 게임 화면을 보여주는 게 아니라 키보드 입력만 전달하기 때문에 검은 창이 나타나는 것이었다. 키보드 입력에도 빠르게 반응했고, 포대 이동과 발사도 정상적으로 확인할 수 있었다.



Q.14 프로토타이핑
컴퓨터화면



Q.14 oled 화면

이번에는 지난번에 실패했던 Q14의 에뮬레이션과 프로토타이핑까지 모두 정상적으로 진행할 수 있었다. 특히 가장 많은 시간을 잡아먹었던 부분은 FPGA 합성이나 RTL 코드가 아니라 usb 연결과 pico 펌웨어 업로드였다. 처음에는 기존 빌드 파일을 정리하고 FPGA와 테스트벤치를 다시 빌드했지만, 여전히 게임 화면이 나타나지 않았다. pico 펌웨어 업로드 과정과 usb 연결 상태를 다시 확인하고 나서야 정상적으로 실행할 수 있었다. 또 에뮬레이션과 프로토타이핑은 같은 FPGA와 pico를 사용하는데도 동작 방식과 실행 속도가 많이 달랐다. 에뮬레이션에서는 systemC 테스트벤치와 usb 통신 때문에 게임이 너무 느려서 키보드로 조작하기도 어려웠지만, 프로토타이핑에서는 systemC 테스트벤치 없이 컴퓨터의 키보드 입력을 pico를 통해 FPGA로 전달하고, FPGA에서 처리한 게임 화면을 oled에 출력해서 실제 게임처럼 조작할 수 있었다. 다음에는 오류가 발생하면 무작정 빌드부터 다시 하기보다는 usb 연결이나 펌웨어 설정부터 확인해 봐야겠다고 생각했다.